LAB 03 – SDD – HTTP Aggregator – abdalmajeed alawad – 992116

1- هدف الوظيفة واختيار المشروع

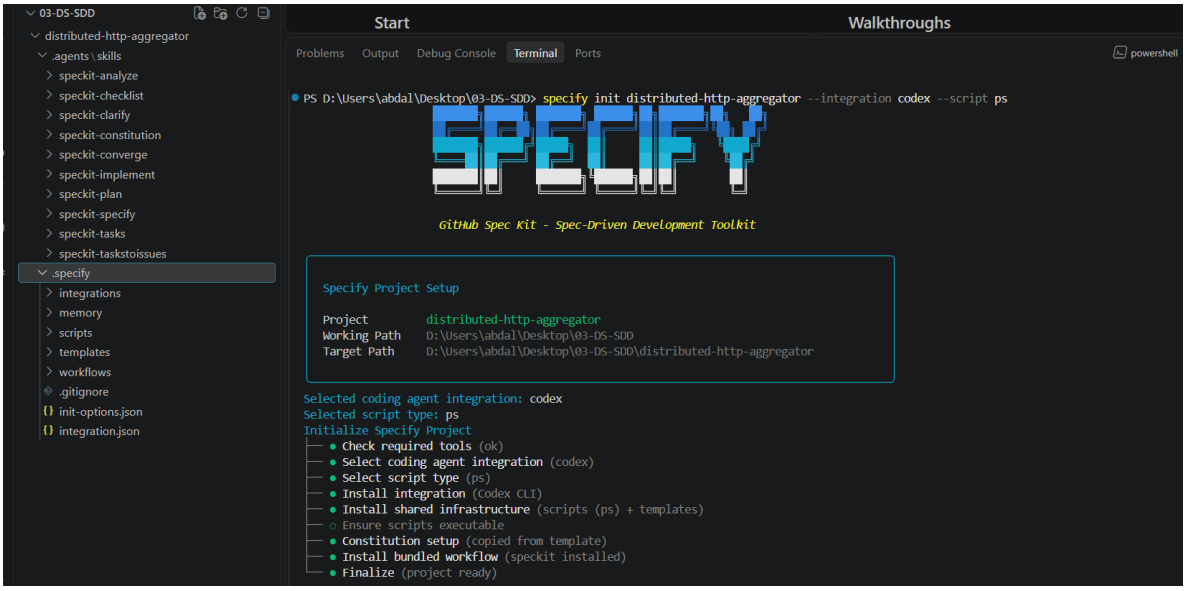
اختير الخيار الثاني من وظيفة عملي أساسيات النظم الموزعة، وهو بناء عميل HTTP غير متزامن ومجمّع نتائج. يرسل النظام طلبات متوازية إلى ثلاثة أو خمسة خوادم مستقلة، ويحتفظ بالاستجابات السليمة حتى عند فشل بعض الخوادم، ثم يصدر تقرير JSON موحداً يصف نتيجة كل خادم وزمنه والزمن الكلي للتنفيذ.

التطبيق في المشروع	المعيار التقني
HttpClient.sendAsync + CompletableFuture	التزامن غير الحاجب
Jetty 12، من 3 إلى 5 Workers	محاكاة الخوادم
HTTP 500 / Timeout / Connection Failure	العزل عند الفشل
ترتيب ثابت وبيانات من الاستجابات الناجحة فقط	التجميع الحتمي
JUnit 5 و Maven clean verify	التحقق الآلي

2- تهيئة المشروع باستخدام GitHub Spec Kit

بدأنا بإنشاء مستودع منظم وفق منهجية Spec-Driven Development، ثم تهيئة GitHub Spec Kit داخل VS Code. أنتجت الأداة مجلدي .specify و .agents/skills، وأصبحت مراحل كتابة الدستور والمواصفات والخطة والمهام والتحليل والتنفيذ قابلة للتتبع في Git.

```
specify init distributed-http-aggregator --integration codex --script ps
```

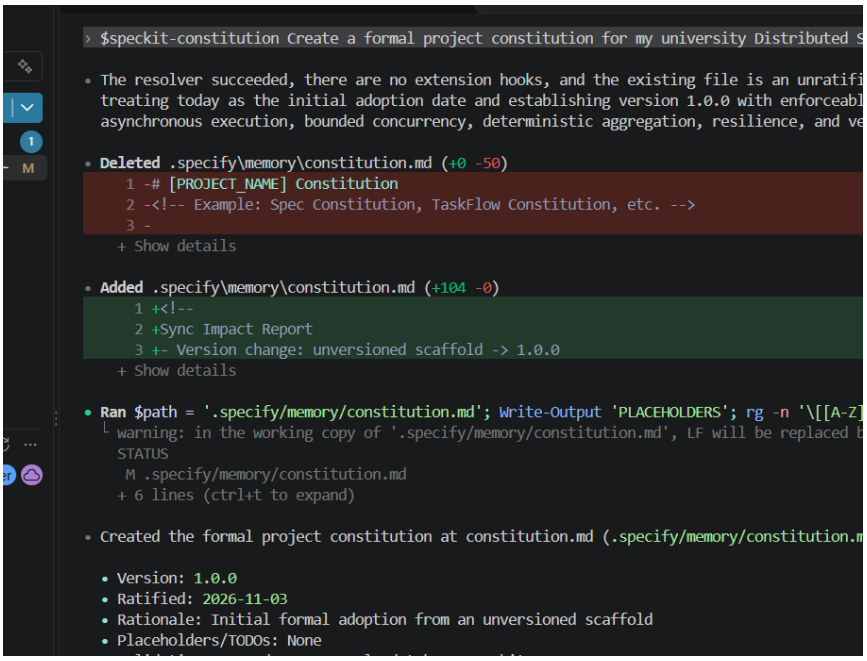


الشكل (1): تهيئة Spec Kit وظهور المجلدات والمهارات الخاصة بدورة عمل SDD.

3- صياغة دستور المشروع (Project Constitution)

وُثّقت القواعد الحاكمة للمشروع في constitution.md قبل كتابة التطبيق: التنفيذ غير الحاجب، ضبط عدد الطلبات المتوازية، المحافظة على الترتيب، عزل فشل الطلبات، وإلزامية الاختبارات. وبعد تثبيت متطلب لغة Java، حُدّث الدستور إلى الإصدار 2.0.0 مع تحديد Java 21 وMaven و5 JUnit و12 Jetty وJackson 2.x.

```
specs: .specify/memory/constitution.md
Java 21 | CompletableFuture | HttpClient.sendAsync()
Maven | Jetty 12 | Jackson 2.x | JUnit 5
```



الشكل (2): إعداد دستور المشروع في المرحلة الأولى وتوثيق قواعد التنفيذ والوثوقية.

4- كتابة المواصفات الوظيفية (Specification)

حُددت المتطلبات الوظيفية FR-001 إلى FR-024 ومعايير النجاح SC-001 إلى SC-009 داخل spec.md. قُسمت الوظائف إلى ثلاث قصص مستخدم: جمع النتائج المتوازية، الاحتفاظ بالنجاح الجزئي عند الأعطال، والتعامل الواضح مع الفشل الكلي والإعدادات غير الصالحة. لكل قصة سيناريوهات قبول قابلة للاختبار.

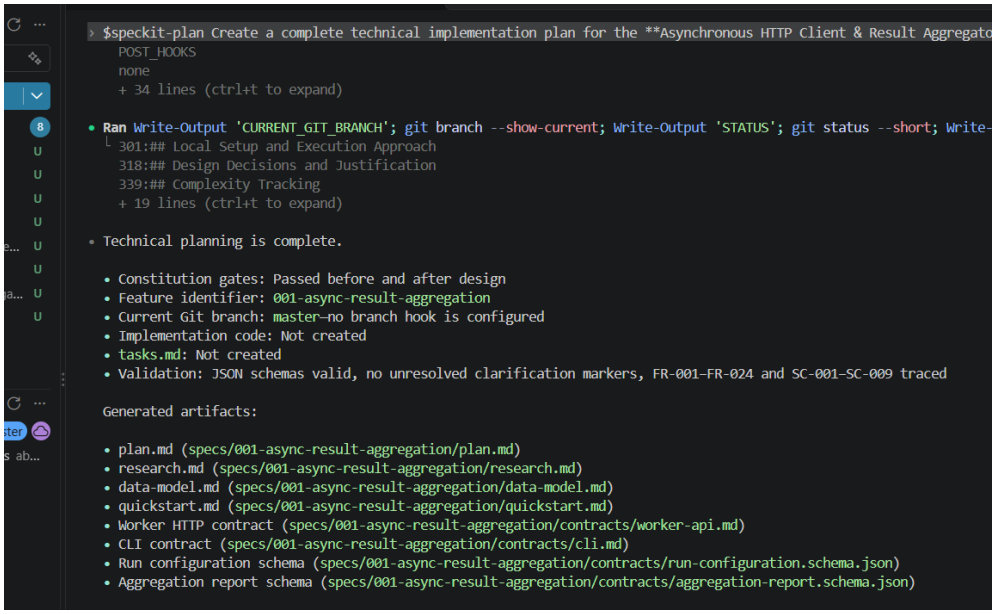
اختبار القبول الرئيسي	قصة المستخدم
نتائج مرتبة وزمن كلي يقارب أبطأ خادم	US1 – النجاح المتوازي
استمرار جمع النتائج السليمة عند HTTP 500 والتوقف والمهلة	US2 – الفشل الجزئي
تقرير صحيح بلا بيانات ناجحة، والتحقق قبل بدء الشبكة	US3 – الفشل الكلي

```
specs/001-async-result-aggregation/spec.md
FR-001 .. FR-024    |    SC-001 .. SC-009
```

5- وضع خطة التنفيذ والتصميم المعماري

حوّلت مرحلة speckit-plan المتطلبات إلى مكونات تقنية واضحة، مع عقود HTTP وCLI ومخططات JSON ونموذج البيانات ودليل التشغيل. بُني التصميم على مبدأ Fan-out / Fan-in: يبدأ العميل الطلبات دون انتظار متسلسل، ثم يجمع النتائج بعد اكتمالها وفق ترتيب الإعداد الأصلي. CompletableFuture.allOf

```
$speckit-plan
plan.md | research.md | data-model.md | quickstart.md
contracts/worker-api.md | contracts/cli.md | contracts/*.schema.json
```

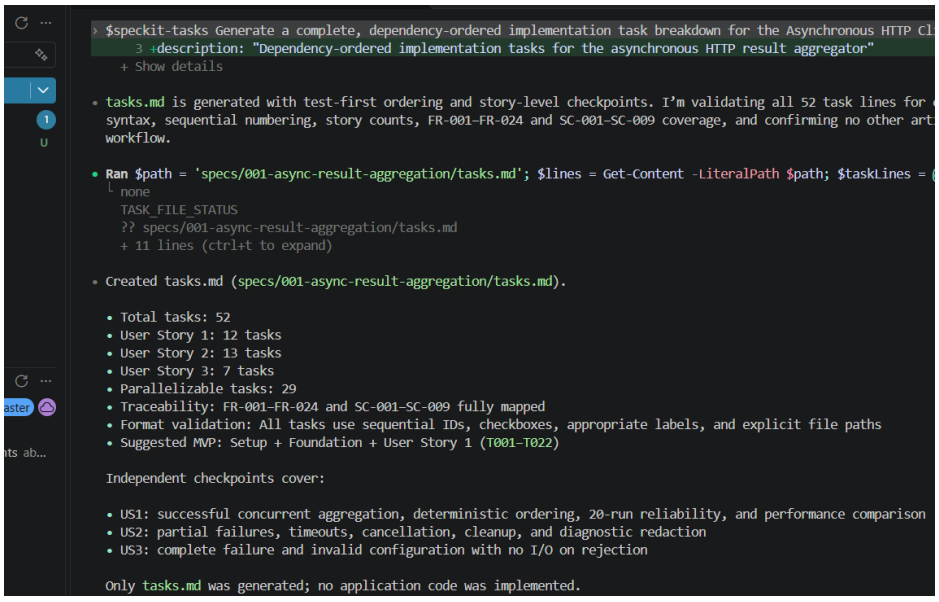


الشكل (3): اكتمال خطة التنفيذ وإنشاء عقود HTTP وCLI ومخططات التحقق.

6- تجزئة الخطة إلى مهام تنفيذية

قسم ملف tasks.md العمل إلى 52 مهمة مرتبة على مراحل الإعداد، النماذج والتحقق، قصص المستخدم الثلاث، التحقق الشامل والتوثيق. صُممت مهام الاختبار لتسبق التنفيذ، مع نقاط فحص Checkpoints بعد كل مجموعة حتى لا يُقبل نجاح ظاهري من الوكيل البرمجي دون إثبات.

```
$speckit-tasks
T001 - T052
Tests First -> Implementation -> Verification
```

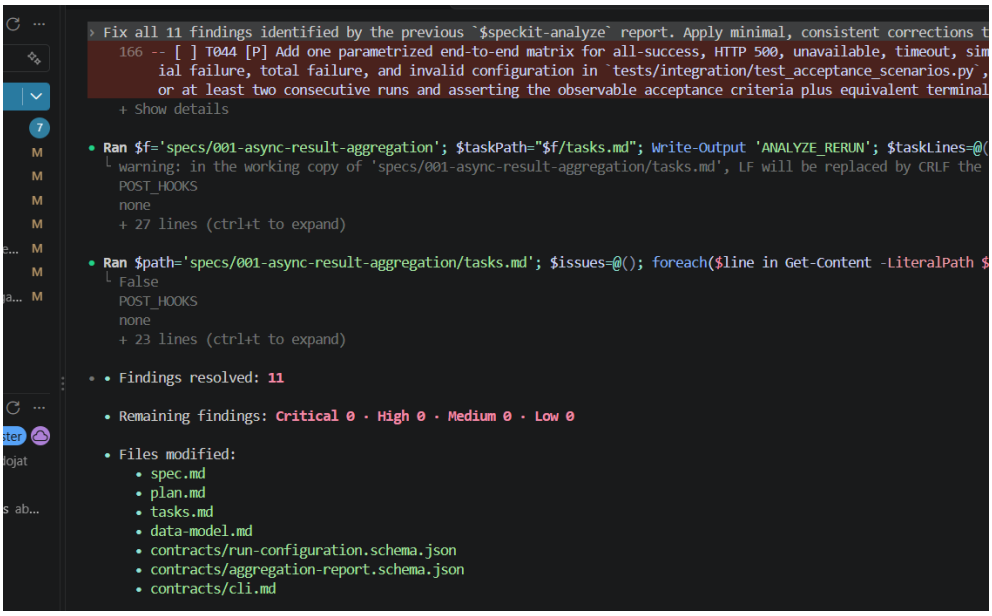


الشكل (4): إعداد هيكل المهام واختبارات القبول ونقاط التحقق المرحلية.

7- تحليل الاتساق وتصحيح الملاحظات

استُخدمت مرحلة speckit-analyze لمقارنة الدستور والمواصفات والخطة والمهام والعقود قبل التوسع بالتنفيذ. أظهر التحليل الأول 11 ملاحظة متفاوتة الأولوية، ثم عُدّلت الوثائق والمخططات وأُعيد التحقق حتى أصبح عدد الملاحظات المتبقية صفراً. يُظهر ذلك دور مراجعة المواصفات قبل كتابة الكود.

```
$speckit-analyze
Findings resolved: 11
Remaining: Critical 0 | High 0 | Medium 0 | Low 0
```



الشكل (5): تصحيح الملاحظات الإحدى عشرة وإعادة تحليل الاتساق دون نتائج متبقية.

8- تنفيذ معمارية Java غير المتزامنة

اعتمد التطبيق النهائي Java 21 حصراً. يشغّل WorkerFleet خوادم Jetty مستقلة، ويرسل WorkerHttpClient طلباته باستخدام sendAsync. تنسق AggregationService الطلبات عبر CompletableFuture وAsyncPermitPool لتقييد التوازي، ثم تُنتج AggregationReport كائناً ثابتاً يحافظ على هوية كل خادم وترتيبه ويُسجل المدد باستخدام System.nanoTime().

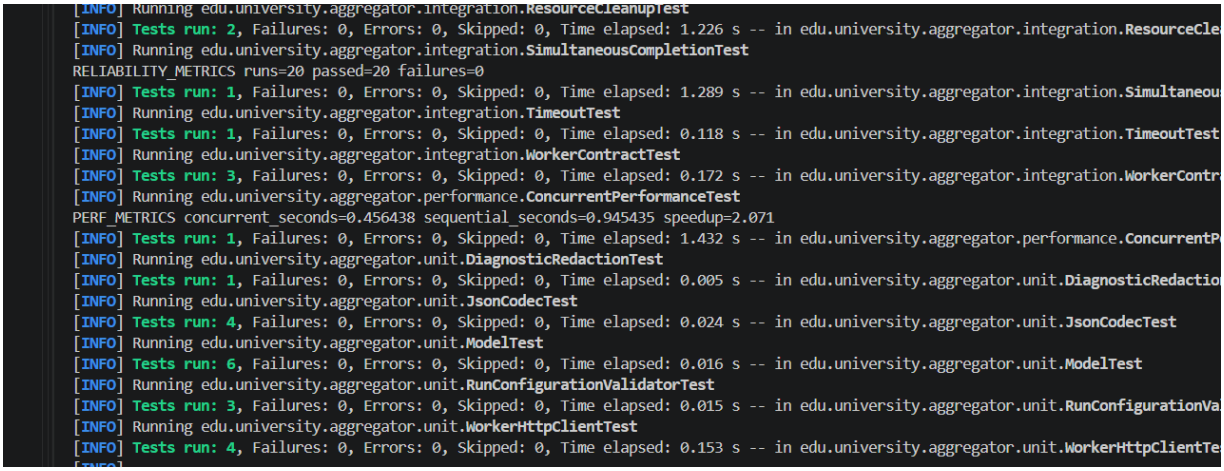
المسؤولية	المكوّن
محاكاة عقد HTTP وتنظيف الموارد	WorkerServer / WorkerFleet
طلبات غير حاظرة والتحقق من الردود وتحديد الأخطاء	WorkerHttpClient
تنظيم المهام، المهلات والتجميع بالترتيب	AggregationService
تحديد سقف التوازي دون تسلسل الطلبات	AsyncPermitPool
JSON صارم وفحص جميع أخطاء الإعداد	JsonCodec / Validator

```
HttpClient.sendAsync(...)
CompletableFuture.allOf(...)
System.nanoTime()
```

9- التحقق الآلي واختبار أداء التزامن

شُغّل Maven مع اختبارات JUnit للتحقق من عقود البيانات، والتعامل مع الأعطال، وترتيب النتائج، وتحرير الموارد. أظهرت اللقطة المسجلة نجاح 51 اختباراً دون إخفاق، ونجاح 20 تجربة إكمال متزامن من أصل 20. في اختبار الأداء نفسه بلغ التنفيذ المتوازي 0.456438 ثانية، مقابل 0.945435 ثانية للتنفيذ التسلسلي، بتسريع 2.071 مرة.

```
.\mvnw.cmd clean verify
PERF_METRICS concurrent_seconds=0.456438 sequential_seconds=0.945435 speedup=2.071
RELIABILITY_METRICS runs=20 passed=20 failures=0
```

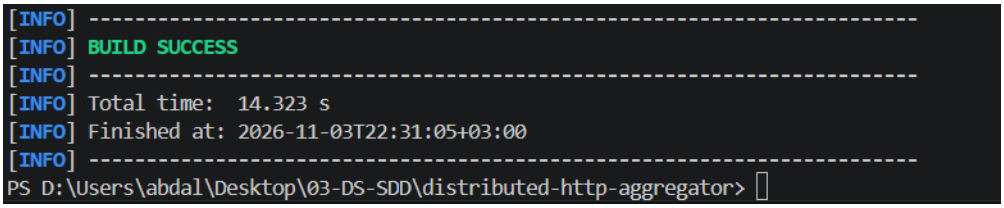


الشكل (6): نجاح اختبارات JUnit والوثوقية وقياس تسريع التنفيذ المتوازي.

10- إنشاء الحزمة التنفيذية والتحقق من نجاح البناء

بعد اكتمال الاختبارات أنشأ Maven ملف JAR يحتوي التبعيات اللازمة للتشغيل. تُظهر لقطة التنفيذ حالة BUILD SUCCESS بعد 14.323 ثانية، وهو دليل على اكتمال مراحل التجميع والاختبار والتغليف في البيئة المحلية.

```
.\mvnw.cmd clean verify
java -jar target\distributed-http-aggregator.jar list-scenarios
```

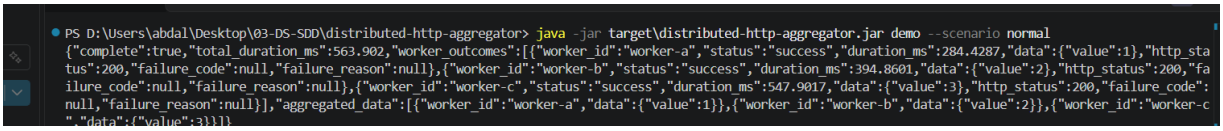


الشكل (7): ظهور BUILD SUCCESS بعد تشغيل Maven clean verify بنجاح.

11- اختبار الحالة الطبيعية (Normal Aggregation)

شُغّلت ثلاثة خوادم تجريبية وأُرسلت إليها الطلبات بالتوازي. أعاد النظام ثلاث نتائج success مع رمز HTTP 200 وبيانات مرتبطة بكل worker_id. يظهر التقرير complete=true ووجود ثلاثة عناصر في aggregated_data، مع المحافظة على ترتيب الخوادم كما في الإعداد.

```
java -jar target\distributed-http-aggregator.jar demo --scenario normal
```

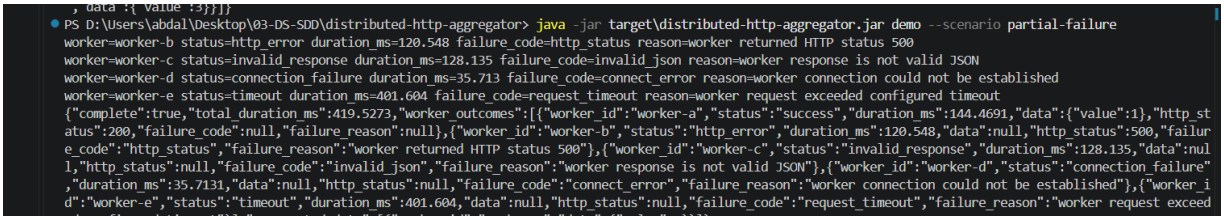


الشكل (8): نجاح الخوادم الثلاثة وصدور تقرير تجميع JSON مرتب.

12- اختبار عزل الأعطال والنجاح الجزئي

جمع سيناريو partial-failure خمس عقد مختلفة: خادم ناجح، وخادم أعاد HTTP 500، واستجابة JSON غير صالحة، وفشل اتصال، وتجاوز مهلة الطلب. لم يتوقف البرنامج بسبب فشل أربع عقد، بل احتفظ ببيانات الخادم السليم في aggregated_data وسجّل سبب الإخفاق لكل عقدة بصورة مستقلة.

```
java -jar target\distributed-http-aggregator.jar demo --scenario partial-failure
```

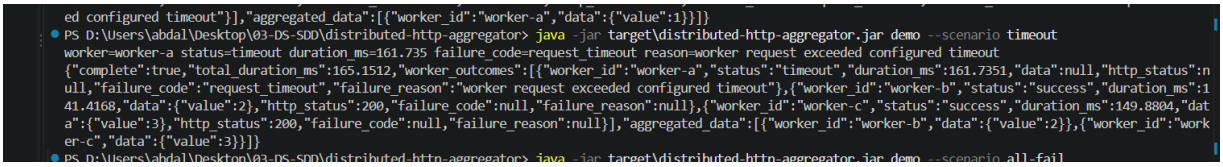


الشكل (9): الاحتفاظ بنتيجة خادم ناجح مع تصنيف أربعة أعطال مستقلة.

13- اختبار المهلة الزمنية المستقلة (Timeout)

في سيناريو timeout تأخرت إحدى العقد عن المهلة المسموحة، فصُنّفت حالتها timeout مع failure_code=request_timeout. في المقابل اكتمل طلبا الخادمين الآخرين بنجاح وأُدرجت بياناتهما في التقرير. تبرهن النتيجة أن انتهاء مهلة عقدة لا يمنع تقدم بقية الطلبات.

```
java -jar target\distributed-http-aggregator.jar demo --scenario timeout
```

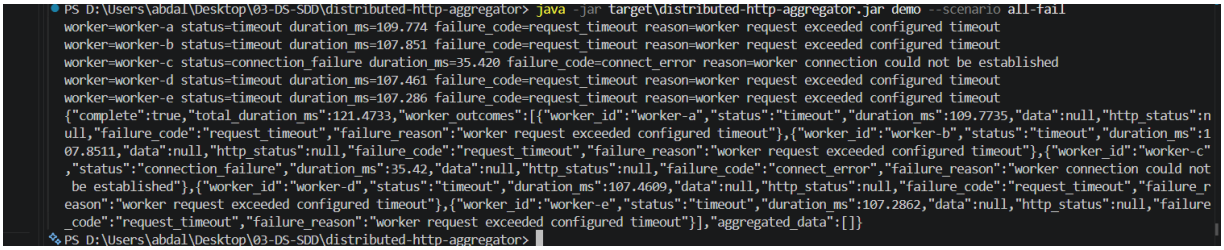


الشكل (10): تصنيف طلب متأخر على أنه Timeout مع استمرار نجاح الخادمين الآخرين.

14- اختبار الفشل الكلي لجميع الخوادم

شُغّل سيناريو all-fail بخمس عقد غير ناجحة. أصدر النظام تقريراً نهائياً مع complete=true، وخمس حالات فشل مسجلة، وقائمة aggregated_data فارغة. يميز هذا السلوك فشل الخوادم الخارجية عن تعطل البرنامج المحلي، ويحافظ على قابلية تحليل النتائج حتى عندما لا يتوفر أي رد ناجح.

```
java -jar target\distributed-http-aggregator.jar demo --scenario all-fail
```

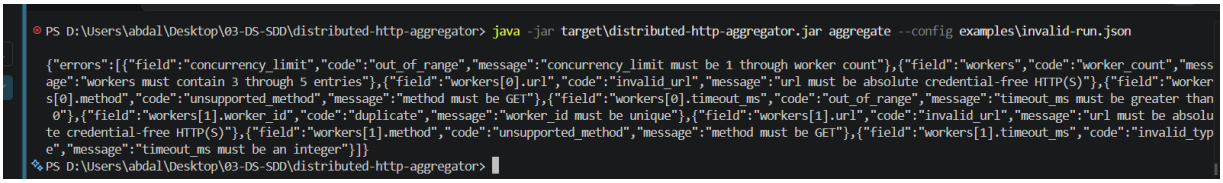


الشكل (11): صدور تقرير صالح بخمس حالات فشل وصفر بيانات مجمعة ناجحة.

15- رفض الإعدادات غير الصالحة قبل تنفيذ أي طلب

اختُبر الملف invalid-run.json الذي يتضمن قيماً غير صالحة للتوازي وعدد العمال والروابط وطريقة الطلب والمهلة وهويات مكررة. أعاد الأمر قائمة منظمة من تسعة أخطاء تحقق، مع رمز خروج 2 ودون تقرير نجاح على stdout. وأثبت اختبار request spy ضمن JUnit عدم إرسال أي طلب شبكة قبل رفض الإعداد غير الصحيح.

```
java -jar target\distributed-http-aggregator.jar aggregate --config examples\invalid-run.json
# Exit code: 2  |  Structured JSON errors
```



الشكل (12): إظهار أخطاء التحقق في JSON عند رفض ملف إعداد غير صالح.

16- تتبع التطوير والتسليم بواسطة Git

حُفظ تسلسل التطوير في مستودع GitHub على مراحل مستقلة. يشمل السجل تهيئة Spec Kit، إنشاء الدستور والمواصفات والخطة، تصحيح التحليل، تأسيس Java، التنفيذ المتوازي، معالجة الأخطاء، الاختبارات النهائية والتوثيق. انتهى العمل بتنظيف الملفات التاريخية غير المطلوبة للتسليم مع الاحتفاظ بسجل Git الكامل.

```
git log --oneline
git status
https://github.com/abdojat/distributed-http-aggregator
```

رسالة Commit موثّقة	محطة التطوير
chore: initialize Spec Kit project	تهيئة سير العمل
docs: establish implementation architecture and contracts	المواصفات والتخطيط
docs: migrate SDD architecture and tasks to Java 21	الانتقال إلى Java
feat: implement asynchronous Java HTTP aggregation and verify US1	تجميع النتائج
,feat: implement Java failure isolation, timeouts and cancellation	الفشل والعزل
chore: finalize Java-only submission and remove legacy Python code	تجهيز التسليم

17- ملخص نتائج الاختبارات النهائية

النتيجة	مؤشر التحقق
51 ناجح، 0 فاشل، 0 أخطاء، 0 متجاهل	JUnit 5
20 / 20 تشغيل ناجح	الوثوقية تحت الإكمال المتزامن
0.456438 ثانية	الزمن المتوازي المقاس
0.945435 ثانية	الزمن التسلسلي المقاس
2.071 مرة	التسريع المحقق
نجاح الحالات الطبيعية والجزئية والكلية والمهل	اختبارات سيناريوهات CLI
تسعة أخطاء منظمة، خروج 2، دون إرسال طلبات	التحقق من الإدخال
FR-001..FR-024 وSC-001..SC-009 موثقة في verification.md	متطلبات المواصفات

أُخذت قيم الزمن من التشغيل الموثق في لقطة الاختبار الأخيرة، لذلك قد تختلف قليلاً عن قياسات التشغيلات السابقة المسجلة في مستند verification.md بسبب تغير ظروف التنفيذ. لم تؤثر الفروق على نتيجة المقارنة أو اجتياز معايير القبول.

18- التقييم الهندسي (Engineering Reflection)

ساعدت منهجية SDD على منع توليد المشروع دفعة واحدة دون معايير واضحة، إذ استُخدمت المواصفات وعقود JSON ومعايير النجاح لتوجيه الوكلاء البرمجيين وفحص مخرجاتهم. وكشف التحليل الاستباقي 11 ملاحظة جرى تصحيحها قبل التوسع بالتنفيذ؛ كما كشفت الاختبارات أخطاء في تصنيف Timeout وحالة سباق عند الإلغاء، فصُححت قبل اعتماد البناء النهائي.

عندما تقرر اعتماد Java 21 كلغة نهائية، جرى تحديث Constitution والخطة والمهام بدلاً من تغيير الكود بمعزل عن المواصفات. احتُفظ بسجل التطوير التاريخي، بينما اقتصرت نسخة التسليم على مصدر Java وJUnit وMaven ووثائق SDD. هذا الفصل يوضح أن دور الوكيل البرمجي كان تنفيذ مهام محددة، وأن قبول النتائج اعتمد على الاختبارات والأدلة الفعلية.

19- الخلاصة والمراجع

أُنجز نموذج عميل HTTP غير متزامن ومجمّع نتائج باستخدام Java 21، مع خوادم HTTP تجريبية مستقلة، ضبط للتوازي، ومهلات محددة لكل طلب، وإخراج JSON حتمي. أثبتت الاختبارات أن فشل بعض الخوادم أو جميعها لا يُفقد نتائج العقد الأخرى ولا يسبب انهيار عملية التجميع، وأن التنفيذ المتوازي أسرع من تنفيذ الطلبات بالتسلسل ضمن تجربة القياس المعتمدة.

مرجع المشروع: https://github.com/abdojat/distributed-http-aggregator

مرجع منهجية التطوير: /https://github.github.com/spec-kit

المراجع الداخلية: README.md وconstitution.md وspec.md وplan.md وtasks.md وverification.md وعقود JSON/HTTP داخل المستودع.